游戏助手服务的调用者开发说明

目录

1 游戏助手服务的调用者开发说明 **1**

2 更新日志 **1**
2.1 2025-05-29 1
2.2 2025-05-09 1

3 开发说明 **1**
3.1 启动进度 1
3.2 运行时信息 2
3.3 服务进程管理 2
3.4 代码例子 2

1 游戏助手服务的调用者开发说明

编辑日期 `2025-05-29` 。
游戏助手服务版本 `2025-05-29` 。

2 更新日志

2.1 2025-05-29

- `服务进程管理` 增加 `父进程` 说明。
- 增加 `运行时信息` 。
- 更新 `代码例子` ，增加 `--uuid` 参数。

2.2 2025-05-09

- 初版。
- 提供 `开发说明` 。

3 开发说明

3.1 启动进度

游戏助手服务是一个控制台程序，单独启动会弹出命令行窗口，同时会有日志打印在窗口中。
如果需要监控服务的启动进度，可以通过读取日志来确定当前进度，具体可查看 `代码例子` 。

您也可以修改配置文件，禁用 `vision` 、`knowledge` 、`asr` 服务模块，这样服务启动时不会加载这些模块，会大幅缩短启动时间，在启动完成后，通过接口去启用它们。具体接口请查阅 `《游戏助手服务接口说明》` 中的 `启停接口` 。

3.2 运行时信息

服务启动完成后，会生成一个 json 文件，里面包含了服务端口号等信息。
请通过传递 `--uuid` 参数来确认是否可以使用该文件。
具体请参考《游戏助手服务 Runtime 配置文件说明》。

3.3 服务进程管理

如果游戏助手服务作为独立进程启动时，调用者应该确保服务进程不会脱离掌控。
比如当调用者意外退出时，服务进程会一直留在后台，占用大量系统资源，带来不好的体验。

游戏助手服务提供了监控调用者进程和父进程的功能，如果调用者进程或父进程意外退出，会在全部任务执行完成后自动退出，以释放系统资源。具体请查阅 《游戏助手服务配置文件说明》 中的 `caller 游戏助手服务调用者` 配置项。

3.4 代码例子

以下是启动游戏助手服务的 C# 代码例子。仅供参考。

```
// 启动游戏助手服务
public void Launch()
{
    // 假定游戏助手服务路径是"< 调用者目录 >\apps\GameAssistantToolServer.exe"
    string AppFolder = Path.Combine(Directory.GetCurrentDirectory(), "apps");
    string ExeFile = Path.Combine(AppFolder, "GameAssistantToolServer.exe");
    string UUID = Guid.NewGuid().ToString();

    Process process = new Process();
    process.StartInfo.FileName = ExeFile;
    process.StartInfo.Arguments = $"--uuid {UUID}";
    process.StartInfo.UseShellExecute = false;
    process.StartInfo.CreateNoWindow = true;

    process.StartInfo.StandardOutputEncoding = Encoding.UTF8;

    process.StartInfo.RedirectStandardInput = true;
    process.StartInfo.RedirectStandardOutput = true;
    process.StartInfo.RedirectStandardError = true;

    process.EnableRaisingEvents = true;
    process.StartInfo.WorkingDirectory = AppFolder;
    process.OutputDataReceived += this.ProcessOutputDataReceived;
    process.ErrorDataReceived += this.ProcessErrorDataReceived;

    process.Start();

    process.BeginOutputReadLine();
    process.BeginErrorReadLine();

    process.WaitForExit();
}

private void ProcessOutputDataReceived(object sender, DataReceivedEventArgs e)
{
    HandleLogs(e.Data);
}

private void ProcessErrorDataReceived(object sender, DataReceivedEventArgs e)
{
    HandleLogs(e.Data);
}
```

```
private void HandleLogs(string content)
{
    // 检查启动完成的关键字
    var SuccessKeywords = new List<string>()
    {
        "is running on http://"
    };
    bool FoundSuccessKeyword = false;
    foreach (string keyword in SuccessKeywords)
    {
        if (content.Contains(keyword))
        {
            Console.WriteLine(" 启动完成");
            FoundSuccessKeyword = true;
            break;
        }
    }

    // 检查服务关闭的关键字
    var ShutdownKeywords = new List<string>()
    {
        "shutdown complete"
    };
    bool FoundShutdownKeyword = false;
    foreach (var keyword in ShutdownKeywords)
    {
        if (content.Contains(keyword))
        {
            Console.WriteLine(" 服务已关闭");
            FoundShutdownKeyword = true;
            break;
        }
    }

    // 启动中
    if (!FoundSuccessKeyword && !FoundShutdownKeyword)
    {
        // 检查启动过程中的关键字
        var keywords = new Dictionary<string, string>
        {
            { "Loaded config", "20%" },
            { "Start initializing vision model", "40%" },
            { "Start initializing llm model", "60%" },
            { "Start initializing asr model", "80%" },
        };

        foreach (var pair in keywords)
        {
            if (content.Contains(pair.Key))
            {
                Console.WriteLine($" 启动进度: {pair.Value}");
            }
        }
    }
}
```